



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- Denominación del Programa de Formación: Análisis y Desarrollo de Software.
- Código del Programa de Formación: 228118
- Nombre del Proyecto Formativo: Construcción de software a la medida para el control de acceso de personal, elementos de almacén y flujo de equipos externos para el centro de gestión agroempresarial del oriente.
- Fase del Proyecto: Ejecución
- Actividad de Proyecto Formativo: CODIFICAR LOS MÓDULOS DEL SISTEMA DE INFORMACIÓN
- Competencia: Construcción del software.
- Resultados de Aprendizaje:
 - MODELAR LAS FUNCIONES DEL SOFTWARE DE ACUERDO CON EL INFORME DE REQUISITOS.
 - DESARROLLAR PROCESOS LÓGICOS A TRAVÉS DE LA IMPLEMENTACIÓN DE ALGORITMOS.
 - CODIFICAR EL SOFTWARE DE ACUERDO CON EL DISEÑO ESTABLECIDO.
- Duración de la Guía de Aprendizaje: 150 horas

2. PRESENTACIÓN

El patrón MVC (Modelo – vista - controlador), permite una clara separación entre los diferentes componentes de una aplicación, en java para escritorio se puede aplicar la misma filosofía dentro de cada package.

El acceso a la base de datos se hace realizando el patrón de diseño Singleton ya que solo se desea tener un único acceso a la base de datos.



3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

- Descripción de la(s) Actividad(es)

3.1 Actividades de reflexión inicial:

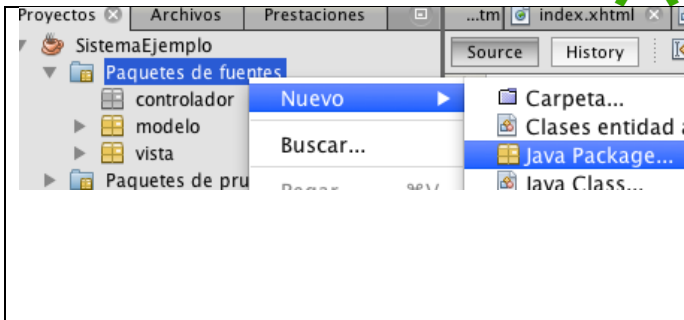
3.1.1 Realizar un formulario para la siguiente clase:

Auto
-idAuto: int; -marcaAuto: String; -modeloAuto: Date; -cantidadPuertas: int; -color: String;
+getIdAuto():String; +setIdAuto(int elId) +getMarcaAuto():String; +setMarcaAuto(String marca) +getCantidadPuertas():int +setCantidadMarcas(int cantidad) +getColor():String +setColor(String elColor) +listar():Iterator +insertar() +modificar() +eliminar()

(Se puede observar JLabel, JComboBox, JButton, JSpinner, JTextField, JTable, JFormattedTextField)

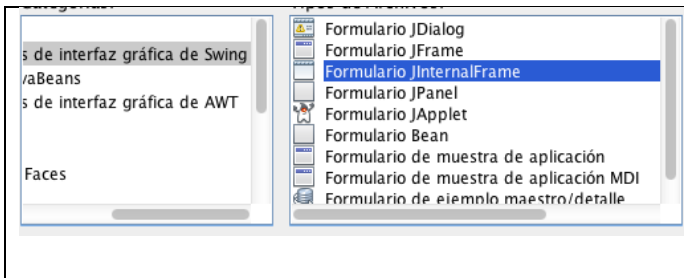
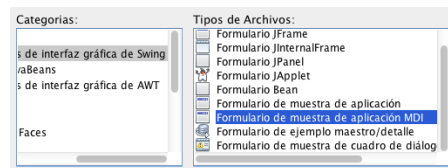
El procedimiento es el siguiente:

Recuerde que primero debe crear un proyecto desde el menú: File > New Project, sin clase principal (quitar el checkmark)



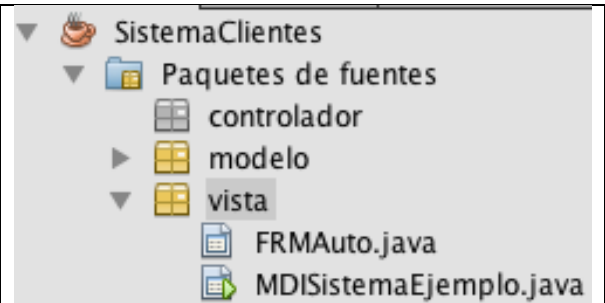
El segundo paso es crear los tres paquetes modelo, vista y controlador; para crear un paquete de clic derecho en la carpeta llamada Source Package > New > Java Package... > escriba vista (recuerde también crea modelo y controlador)

El tercer paso es insertar un MDI dentro del paquete vista con File > New File > Swing GUI Forms > MDI Sample Form; El nombre del MDI debe iniciar con las letras MDI además, fíjese que donde dice package quede seleccionado vista.



De la misma forma agregar un JInternalFrame dentro del paquete vista, de clic derecho sobre el paquete vista > New > Other... > Swing GUI Forms > JInternalFrame Form, esta vez el nombre debe comenzar con FRM

La estructura debe quedar algo similar al pantallazo que aparece:



Tip: Si creó el MDI o el FRM en otro package equivocado o en el source package puede moverlos dando clic derecho sobre el archivo > Refactor > Move... > Selecciona el package correcto.

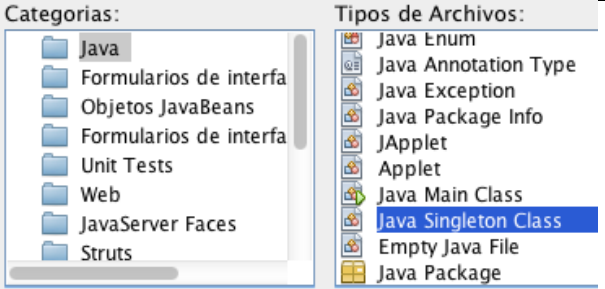
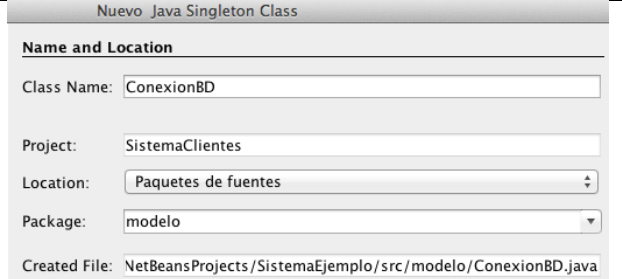
Hasta el momento tenemos tres paquetes, y dentro del paquete vista un MDI y un FRM, en los proyectos debería haber un solo MDI y varios FRM; generalmente cada FRM podría ser una tabla de su base de datos.



De doble clic sobre el FRM creado y agregue los componentes que se piden al inicio de esta actividad (3.1.1)

Si ejecuta la aplicación por primera vez Netbeans le pedirá que seleccione la clase principal, donde seleccionaremos la única que aparece: MDISistemaClientes

3.1.2 Inicie su servidor de base de datos puede ser MySQL a través de XAMPP (o desde MySQLWorkbench) y cree una base de datos llamado sistemaclientes con una tabla llamada Clientes y 5 campos (idCliente SMALLINT AUTOINCREMENT, nombreCliente VARCHAR(50), documentoCliente VARCHAR(15), direccionCliente VARCHAR(50), celularCliente VARCHAR(13)), luego INSERTe 5 clientes.

Dentro del paquete modelo vamos a agregar una conexión a la base de datos usando Singleton así: clic derecho sobre el package modelo > New > Other... > Java > Java Singleton	
	Como nombre colocamos ConexionBD, fíjese que el package sea modelo

Luego aparece la plantilla de la clase singleton lista para codificar entonces escribimos el siguiente código:



```
public class ConexionBD {

    public static Connection conexion;

    private ConexionBD() {
        try {
            String driverBD = "com.mysql.jdbc.Driver";
            String urlBD = "jdbc:mysql://localhost/SistemaEjemplo";
            String usuarioBD = "root";
            String claveBD = "";
            Class.forName(driverBD);
            conexion = DriverManager.getConnection(urlBD, usuarioBD, claveBD);
        } catch (ClassNotFoundException ex) {
            System.err.println("No encuentro el driver:"+ex.getMessage());
        } catch (SQLException ex) {
            System.err.println("Error al conectarme:"+ex.getMessage());
        }
    }

    public static void desconectar() {
        try {
            conexion.close();
        } catch (SQLException ex) {
            System.err.println("Error al desconectarme:"+ex.getMessage());
        }
    }

    public static ConexionBD getInstance() {
        return ConexionBDHolder.INSTANCE;
    }

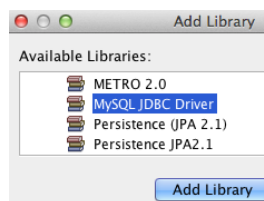
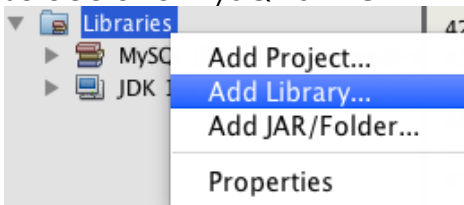
    private static class ConexionBDHolder {

        private static final ConexionBD INSTANCE = new ConexionBD();
    }
}
```

Tip: Un proyecto debería tener una sola ConexionBD dentro del package modelo.

3.1.3 Cargar el driver conector de la base de datos:

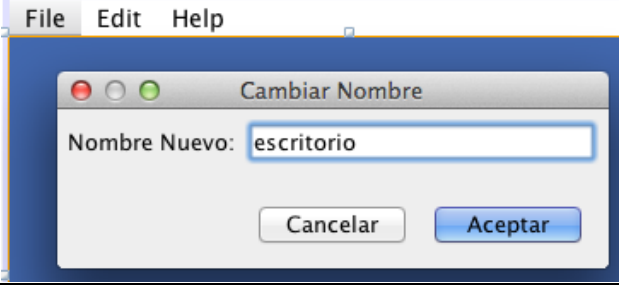
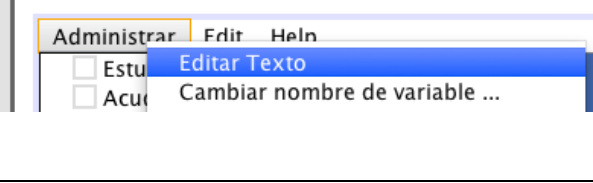
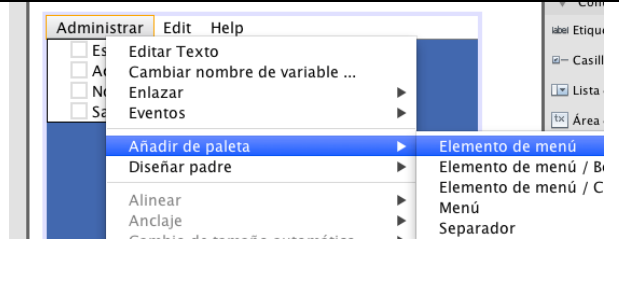
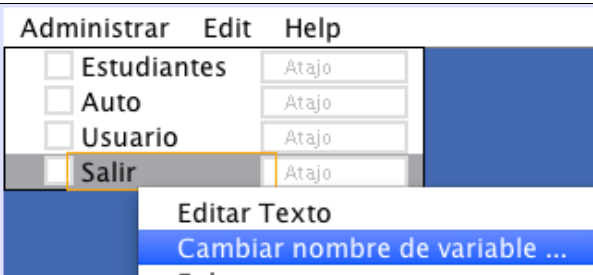
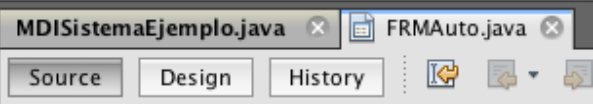
De clic derecho sobre el paquete Libraries, luego de clic en Add Library..., seleccione MySQL JDBC





3.2 Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje:

3.2.1 Configurar el MDI de la siguiente manera:

Para poder visualizar el formulario inicialmente debemos renombrar el fondo negro que aparece en el MDI, para ello de doble clic en el MDI y luego clic derecho sobre el fondo negro > Change Variable Name... > escritorio.	
	Renombramos el menú File dando un solo clic en el menú File y de luego clic derecho > Edit Text.. > Administrar y luego nuevamente clic derecho y Change Variable Name > MNUAdministrar
Agregamos más ítems al menú entre ellos uno que se llame Auto dando clic derecho sobre el menú Administrar > Add from pallette > Menu Item. Damos clic sobre el nuevo Menu Item creado y con clic derecho > Edit Text > Auto, de igual manera Change Variable Name... > ItemAuto.	
	De igual manera cambiamos el texto del menú que dice "Exit" a Salir y el nombre de variable a itemSalir
	Procedemos a codificar el MDI para ello damos clic en Source, si deseamos volver a ver el formulario damos clic en Desing.



Para que podamos usar los FRM debemos agregarlos al MDI como atributos en este caso solo agregamos el formulario creado.

```
public class MDISistemaEjemplo extends javax.swing.JFrame {  
  
    FRMAuto fAuto;  
    FRMEstudiante fEstudiante;  
    FRMPProducto fProducto;  
    FRMCCliente fCliente;  
    FRMPProveedor fProveedor;  
    FRMVenta fVenta;  
    FRMLLogin fLogin;  
    FRMRol fRol;  
    FRMUsuario fUsuario;  
    FRMAuditoria fAuditoria;  
    //aquí pongo el resto de formularios
```

```
/** Crea un nuevo formulario MDISistemaEjemplo */  
public MDISistemaEjemplo() {  
    ConexionBD.getInstance();  
    initComponents();  
    fAuto = new FRMAuto();  
    fEstudiante = new FRMEstudiante();  
    fProducto = new FRMPProducto();  
    fCliente = new FRMCCliente();  
    fProveedor = new FRMPProveedor();  
    fVenta = new FRMVenta();  
    fLogin = new FRMLLogin();  
    fRol = new FRMRol();  
    fUsuario = new FRMUsuario();  
    fAuditoria = new FRMAuditoria();  
    //aquí pongo el resto de formularios
```

Agregamos a dentro del constructor (que ya esta creado) la conexión a la base de datos antes del initComponents(), luego instanciamos todos los formularios.

Ahí mismo, justo debajo de los formularios instanciados agregamos al escritorio los mismos mediante el método add

```
    //aquí pongo el resto de formularios  
    escritorio.add(fAuto);  
    escritorio.add(fEstudiante);  
    escritorio.add(fProducto);  
    escritorio.add(fCliente);  
    escritorio.add(fProveedor);  
    escritorio.add(fVenta);  
    escritorio.add(fLogin);  
    escritorio.add(fRol);  
    escritorio.add(fUsuario);  
    escritorio.add(fAuditoria);  
    //aquí agrego el resto de formularios  
}
```

```
private void itemSalirActionPerformed(java.awt.event.ActionEvent evt) {  
    ConexionBD.desconectar();  
    try {  
        fEstudiante.setClosed(true);  
        fAuto.setClosed(true);  
        fProducto.setClosed(true);  
        fCliente.setClosed(true);  
        fProveedor.setClosed(true);  
        fVenta.setClosed(true);  
        fLogin.setClosed(true);  
        fRol.setClosed(true);  
        fUsuario.setClosed(true);  
        fAuditoria.setClosed(true);  
    } catch (PropertyVetoException ex) {  
        System.err.println("Error al salir:" + ex.getMessage());  
    }  
    System.exit(0);  
}
```

A continuación ir al modo “Desing” menu Administrar y dar doble clic en el menú Salir y programar la acción para desconectar la base de datos, cerrar los formularios y después la aplicación.

```
private void itemEstudiantesActionPerformed(java.awt.event.ActionEvent evt) {  
    fEstudiante.setVisible(true);  
}  
  
private void itemUsuarioActionPerformed(java.awt.event.ActionEvent evt) {  
    fUsuario.setVisible(true);  
}  
  
private void itemAutoActionPerformed(java.awt.event.ActionEvent evt) {  
    fAuto.setVisible(true);  
}
```

Luego dar doble clic en el ítem del menú Auto y codificar la acción para poner visible el formulario.

Tip: Para codificar un login debería poner todos los ítems de los menus con enabled(false) desde el panel Properties, excepto el ítem Salir.

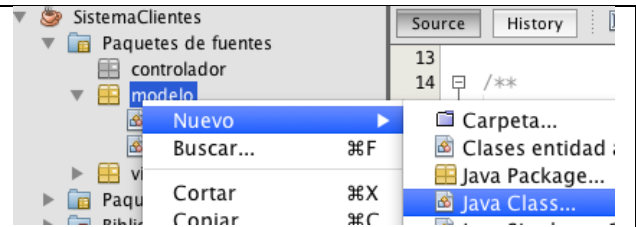


3.3 Actividades de apropiación:

3.3.1 Codificar el modelo Cliente con constructor vacío, con los mutators (getters y setters), esto debe hacerlo dentro del paquete modelo.

Crear la entity class Cliente.java dentro del package modelo dando clic derecho sobre este paquete > New > Java Class...

Poner como nombre Cliente (fíjese que la "C" de Cliente quede en mayúscula) y que el package sea modelo.



```
* @author ceoyear
*/
public class Cliente {

    private int idCliente;
    private String nombreCliente;
    private String documentoCliente;
    private String direccionCliente;
    private String celularCliente;
```

Navegar
Mostrar Javadoc
Encontrar usos
Jerarquía de llamadas
Insertar código...
Reparar importaciones
Reestructurar
Formato

Escribimos los atributos de la clase con el respectivo tipo de dato String, int, Date, ... y luego damos Enter y clic derecho donde quedo parpadeando el curso y seleccionamos Insert Code...

A continuación damos clic en Getter and Setter...

Y chequeamos todos los campos.
Finalmente dar clic en Generar

```
public class Cliente {

    private int idCliente;
    private String nombreCliente;
    private String documentoCliente;
    private String direccionCliente;
    private String celularCliente;
```

Generar
Constructor...
Logger...
Getter...
Setter...
Getter y Setter...

Se crearan los getters y setters:



```

public int getIdCliente() {
    return idCliente;
}

public void setIdCliente(int idCliente) {
    this.idCliente = idCliente;
}

public String getNombreCliente() {
    return nombreCliente;
}

public void setNombreCliente(String nombreCliente) {
    this.nombreCliente = nombreCliente;
}

public String getDocumentoCliente() {
    return documentoCliente;
}

public void setDocumentoCliente(String documentoCliente) {
    this.documentoCliente = documentoCliente;
}

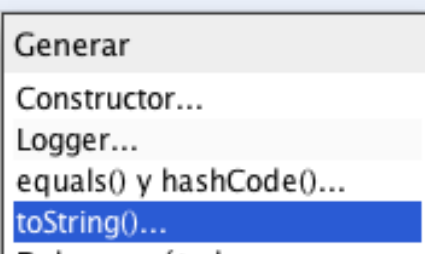
public String getDireccionCliente() {
    return direccionCliente;
}

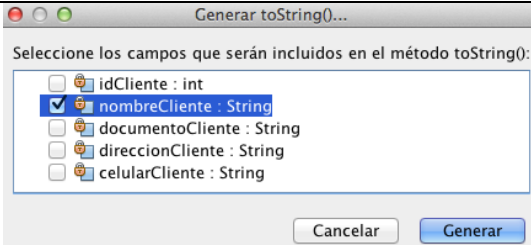
public void setDireccionCliente(String direccionCliente) {
    this.direccionCliente = direccionCliente;
}

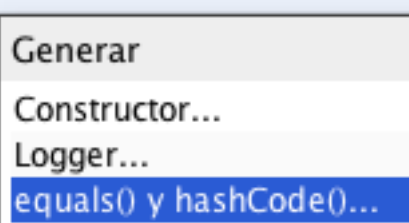
public String getCelularCliente() {
    return celularCliente;
}

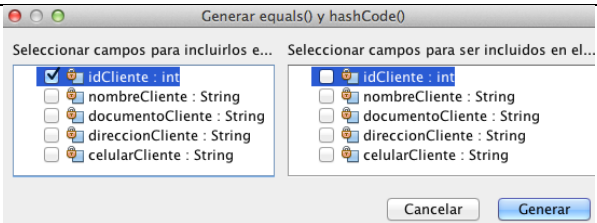
public void setCelularCliente(String celularCliente) {
    this.celularCliente = celularCliente;
}

```

	Insertamos el método toString() el cual es útil cuando queramos mostrar un dato representativo de la clase en una tabla o en un JComboBox, nos ubicamos justo después del último método y damos clic derecho > Insert Code... > toString()...
---	--

Solo seleccionamos el dato representativo en este caso puede ser el nombreCliente	
--	--

	De igual manera agregamos el equals() clic derecho sobre un espacio en blanco del código > Insert Code... > equals() and hashCode()
---	--

Solo se checkea el idCliente para el equals, el cual nos permitira realizar comparaciones basado en el idCliente luego clic en Generar. Esto es útil a la hora de realizar búsquedas por ejemplo cuando seleccionemos un	
---	--



Item de un JComboBox desde el código.

3.3.2 Codificar los metodos CRUD dentro de misma la clase Cliente

```

public Iterator<Cliente> listar(){
    ArrayList<Cliente> losClientes = new ArrayList<>();
    try {
        PreparedStatement sql = ConexionBD.conexion.prepareStatement("SELECT * FROM "+this.getClass().getSimpleName());
        ResultSet rs = sql.executeQuery();
        Cliente unCliente;
        while (rs.next()) {
            unCliente = new Cliente();
            unCliente.setIdCliente(rs.getInt("idCliente"));
            unCliente.setNombreCliente(rs.getString("nombreCliente"));
            unCliente.setDocumentoCliente(rs.getString("documentoCliente"));
            unCliente.setDireccionCliente(rs.getString("direccionCliente"));
            unCliente.setCelularCliente(rs.getString("celularCliente"));
            losClientes.add(unCliente);
        }
    } catch (SQLException ex) {
        System.err.println("Error al listar estudiante:"+ex.getMessage());
    }
    if (losClientes.isEmpty()){
        Cliente miCliente = new Cliente();
        miCliente.setNombreCliente("No hay clientes");
        losClientes.add(miCliente);
    }
    return losClientes.iterator();
}

public void insertar(){
    try {
        PreparedStatement sql = ConexionBD.conexion.prepareStatement("INSERT INTO "
            +this.getClass().getSimpleName()+" VALUES(NULL,?,?,?,?)");
        sql.setString(1, this.getNombreCliente());
        sql.setString(2, this.getDocumentoCliente());
        sql.setString(3, this.getDireccionCliente());
        sql.setString(4, this.getCelularCliente());
        sql.executeUpdate();
        System.out.println(this.getClass().getSimpleName()+ " insertado correctamente");
    } catch (SQLException ex) {
        System.err.println("Error al insertar "+this.getClass().getSimpleName()+":"+ex.getMessage());
    }
}

public void modificar(){
    try {
        PreparedStatement sql = ConexionBD.conexion.prepareStatement("UPDATE "+this.getClass().getSimpleName()+
            " SET nombreCliente = ?, documentoCliente = ?, direccionCliente= ?, celularCliente = ? WHERE idCliente = ?");
        sql.setString(1, this.getNombreCliente());
        sql.setString(2, this.getDocumentoCliente());
        sql.setString(3, this.getDireccionCliente());
        sql.setString(4, this.getCelularCliente());
        sql.setInt(5, this.getIdCliente());
        sql.executeUpdate();
        System.out.println(this.getClass().getSimpleName()+ " modificado correctamente");
    } catch (SQLException ex) {
        System.err.println("Error al modificar "+this.getClass().getSimpleName()+":"+ex.getMessage());
    }
}

```



```
public void eliminar(){
    try {
        PreparedStatement sql = ConexionBD.conexion.prepareStatement("DELETE FROM "
            +this.getClass().getSimpleName()+" WHERE idCliente = ?");
        sql.setInt(1, this.getIdCliente());
        sql.executeUpdate();
        System.out.println(this.getClass().getSimpleName()+ " eliminado correctamente");
    } catch (SQLException ex) {
        System.err.println("Error al eliminar "+this.getClass().getSimpleName()+": "+ex.getMessage());
    }
}
```

```
public Iterator<Cliente> buscar(String busqueda){
    ArrayList<Cliente> losClientes = new ArrayList<>();
    try {
        PreparedStatement sql = ConexionBD.conexion.prepareStatement("SELECT * FROM "+this.getClass().getSimpleName()
            +" WHERE nombreCliente LIKE ? OR documentoCliente LIKE ? OR direccionCliente LIKE ? OR celularCliente LIKE ?");
        sql.setString(1,"%"+busqueda+"%");
        sql.setString(2,"%"+busqueda+"%");
        sql.setString(3,"%"+busqueda+"%");
        sql.setString(4,"%"+busqueda+"%");
        ResultSet rs = sql.executeQuery();
        Cliente unCliente;
        while (rs.next()) {
            unCliente = new Cliente();
            unCliente.setIdCliente(rs.getInt("idCliente"));
            unCliente.setNombreCliente(rs.getString("nombreCliente"));
            unCliente.setDocumentoCliente(rs.getString("documentoCliente"));
            unCliente.setDireccionCliente(rs.getString("direccionCliente"));
            unCliente.setCelularCliente(rs.getString("celularCliente"));
            losClientes.add(unCliente);
        }
    } catch (SQLException ex) {
        System.err.println("Error al buscar "+this.getClass().getSimpleName()+": "+ex.getMessage());
    }
    return losClientes.iterator();
}
```

//El siguiente codigo es útil para llenar el JTable cuando tenemos una llaves foráneas (o JComboBox)

```
public Cliente buscarPorId(int elId){
    Cliente unCliente=new Cliente();
    unCliente.setNombreCliente("Cliente no existe");
    try {
        PreparedStatement sql = ConexionBD.conexion.prepareStatement("SELECT * FROM "+
            this.getClass().getSimpleName()+" WHERE idCliente = ?");
        sql.setInt(1,elId);
        ResultSet rs = sql.executeQuery();
        while (rs.next()) {
            unCliente.setIdCliente(rs.getInt("idCliente"));
            unCliente.setNombreCliente(rs.getString("nombreCliente"));
            unCliente.setDocumentoCliente(rs.getString("documentoCliente"));
            unCliente.setDireccionCliente(rs.getString("direccionCliente"));
            unCliente.setCelularCliente(rs.getString("celularCliente"));
        }
    } catch (SQLException ex) {
        System.err.println("Error al buscar por Id "+ex.getMessage());
    }
    return unCliente;
}
```



Finalmente recuerde que debe cerrar toda la clase con el corchete que cierra.

3.3.3 Crear la clase ControladorCliente dentro del paquete controlador

Para ello cree una nueva Java Class (File > New File > Java > Java Class)

Y cree un método llamado controlarAccion como el que aparece en el siguiente pantallazo

```
public void controlarAccion(ActionEvent evento, Cliente unCliente){
    String accion=evento.getActionCommand();
    switch(accion){
        case "Insertar":
            unCliente.insertar();
            break;
        case "Modificar":
            unCliente.modificar();
            break;
        case "Eliminar":
            unCliente.eliminar();
            break;
    }
}
```

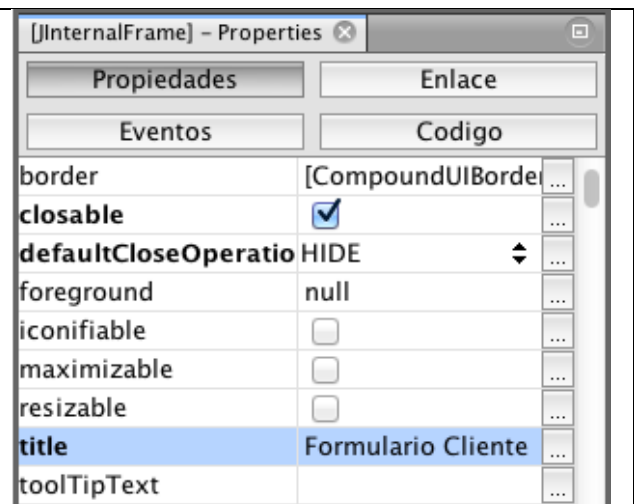
3.4 Actividades de Transferencia el Conocimiento:

Cree la vista para la clase Cliente.

Dentro del paquete vista cree un JFrame Form que llamara FRMCliente

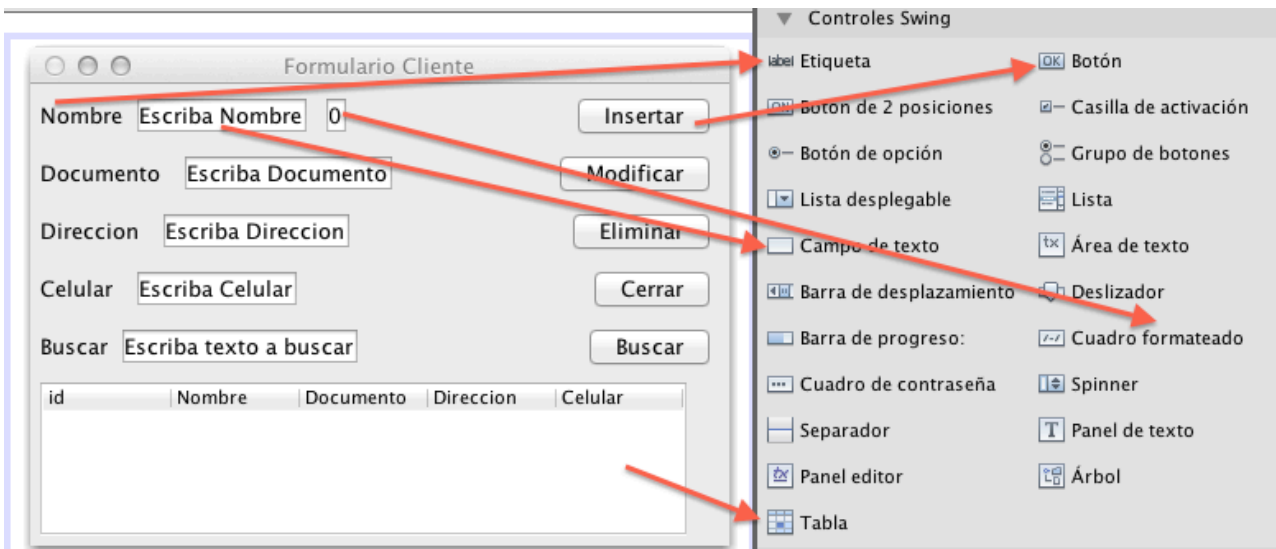
Luego en el panel Properties buscara y ajustara el valor de:
closable a true (que este con el checkmark), defaultCloseOperation a HIDE,
title con "Formulario Cliente"

Closable significa que aparezca el botón de cerrar (en la esquina de la ventana) .





Ahora genere un formulario como el siguiente, a medida que va insertando componentes debe ponerle nombre (variable name) para los JLabel debe iniciar con LBL por ej. LBLNombre, LBLDocumento, para las JTextFields y JFormattedTextField debe iniciar con TXT por. TXTIdCliente, TXTNombreCliente, todos los datos numéricos debe ponerles el texto de 0 (cero); para los JButton debe iniciar con BTN ej. BTNInsertar, BTNCerrar, BTNBuscar, las Tablas con TBL ej. TBLCliente



A continuación se muestran los nombres de los componentes:

label LBLNombre [JLabel]	☐ TXTIdCliente [JFormattedTextField]	☐ BTNInsertar [JButton]
label LBLDireccion [JLabel]	☐ TXTNombreCliente [JTextField]	☐ BTNModificar [JButton]
label LBLDocumento [JLabel]	☐ TXTDocumentoCliente [JTextField]	☐ BTNEliminar [JButton]
label LBLCelular [JLabel]	☐ TXTDireccionCliente [JTextField]	☐ BTNCerrar [JButton]
label LBLBuscar [JLabel]	☐ TXTCelularCliente [JTextField]	☐ BTNBuscar [JButton]
	☐ TXTBuscarCliente [JTextField]	

Ahora agregamos el controladorCliente justo debajo del public class y encima del constructor; Adicionalmente dentro del constructor después del initComponents() le colocamos un valor al Id y lo ocultamos

```
public class FRMCliente extends javax.swing.JFrame {

    ControladorCliente clienteController = new ControladorCliente();

    /** Crea un nuevo formulario FRMEstudiante */
    public FRMCliente() {
        initComponents();
        TXTIdCliente.setValue(0);
        TXTIdCliente.setVisible(false);
        limpiarFormulario();
    }
}
```



```
private void limpiarFormulario() {  
    TXTIdCliente.setValue(0);  
    TXTNombreCliente.setText("Escriba Nombre");  
    TXTDocumentoCliente.setText("Escriba Documento");  
    TXTDireccionCliente.setText("Escriba Direccion");  
    TXTCelularCliente.setText("Escriba Celular");  
    TXTBuscarCliente.setText("Escriba texto a buscar");  
    BTNModificar.setEnabled(false);  
    BTNEliminar.setEnabled(false);  
    llenarTabla();  
}
```

Ahora al final del código del formulario agregamos el código del pantallazo.

Que inicializa cajas de texto, pone botones deshabilitados y vuelve a llenar la tabla.

Al final del código del formulario agregamos el código del pantallazo.

```
public void llenarTabla(){  
    Cliente unCliente = new Cliente();  
    DefaultTableModel tabla = (DefaultTableModel)TBLCliente.getModel();  
    Iterator<Cliente> itCliente = unCliente.listar(); //listado de clientes  
    Object[] filaCliente = new Object[TBLCliente.getColumnCount()]; //una fila  
    tabla.setRowCount(0); //vaciar tabla  
    while (itCliente.hasNext()) { //llenamos cada fila  
        unCliente = itCliente.next();  
        filaCliente[0] = unCliente.getIdCliente();  
        filaCliente[1] = unCliente.getCedulaCliente();  
        filaCliente[2] = unCliente.getNombreCliente();  
        filaCliente[3] = unCliente.getDireccionCliente();  
        filaCliente[4] = unCliente.getCorreoCliente();  
        tabla.addRow(filaCliente); //agregamos la fila a la tabla  
    }  
}
```

```
private void BTNCerrarActionPerformed(java.awt.event.ActionEvent evt) {  
    doDefaultCloseAction();  
}
```

Volvemos al modo "Desing" y damos doble clic en el botón Cerrar y codificamos lo siguiente
doDefaultCloseAction();
(ojo!, no repita lo que esta sobreado con gris)

Nuevamente volvemos al modo Desing y doble clic en el botón de Insertar, en el botón Modificar y en el botón Eliminar y agregue el código que aparece dentro de cada método.

```
private void BTNInsertarActionPerformed(java.awt.event.ActionEvent evt) {  
    clienteController.controlarAccion(evt, obtenerCliente());  
    limpiarFormulario();  
}
```

```
private void BTNModificarActionPerformed(java.awt.event.ActionEvent evt) {  
    clienteController.controlarAccion(evt, obtenerCliente());  
    limpiarFormulario();  
}
```

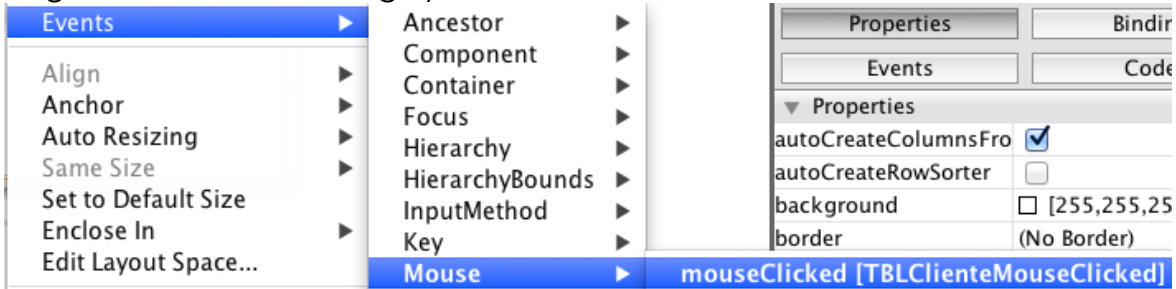
```
private void BTNEliminarActionPerformed(java.awt.event.ActionEvent evt) {  
    clienteController.controlarAccion(evt, obtenerCliente());  
    limpiarFormulario();  
}
```




```
private Cliente obtenerCliente(){
    Cliente elCliente = new Cliente();
    elCliente.setIdCliente((Integer) TXTIdCliente.getValue());
    elCliente.setNombreCliente(TXTNombreCliente.getText());
    elCliente.setDocumentoCliente(TXTDocumentoCliente.getText());
    elCliente.setDireccionCliente(TXTDireccionCliente.getText());
    elCliente.setCelularCliente(TXTCelularCliente.getText());
    return elCliente;
}
```

Ahora agregue método del pantallazo al final de la clase.

Regrese al modo “Desing” y Clic derecho sobre la Tabla > Events > mouseClicked



Codifique dentro del método recién creado el siguiente código

```
private void TBLClienteMouseClicked(java.awt.event.MouseEvent evt) {
    if(evt.getClickCount()==2){
        int fila = TBLCliente.rowAtPoint(evt.getPoint());
        if(fila>=1){
            TXTId.setText(TBLCliente.getValueAt(fila, 0).toString());
            TXTNombre.setText(TBLCliente.getValueAt(fila, 1).toString());
            TXTDireccion.setText(TBLCliente.getValueAt(fila, 2).toString());
            TXTCorreo.setText(TBLCliente.getValueAt(fila, 3).toString());
            TXTCedula.setText(TBLCliente.getValueAt(fila, 4).toString());
            BTNModificar.setEnabled(true);
            BTNEliminar.setEnabled(true);
        }
    }
}
```

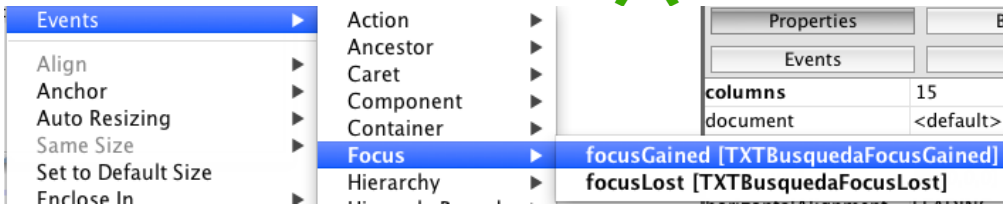
```
private void BTNBuscarActionPerformed(java.awt.event.ActionEvent evt) {
    llenarTablaConBusqueda(TXTBusqueda.getText());
}
```

Volvemos al modo “Desing” y escribimos el código que aparece en el pantallazo

Al final de la clase agregamos el siguiente método con el respectivo código que aparece en el pantallazo

```
private void llenarTablaConBusqueda(String busqueda) {
    Cliente unCliente = new Cliente();
    DefaultTableModel tabla = (DefaultTableModel)TBLCliente.getModel();
    Iterator<Cliente> itClientes = unCliente.buscar(busqueda);
    Object[] filaCliente = new Object[TBLCliente.getColumnCount()];
    tabla.setRowCount(0); //limpia la tabla
    while (itClientes.hasNext()) {
        unCliente = itClientes.next();
        filaCliente[0] = unCliente.getId();
        filaCliente[1] = unCliente.getNombre();
        filaCliente[2] = unCliente.getDireccion();
        filaCliente[3] = unCliente.getCorreo();
        filaCliente[4] = unCliente.getCedula();
        ((DefaultTableModel)TBLCliente.getModel()).addRow(filaCliente);
    }
}
```

Regrese al modo “Desing” y Clic derecho sobre la caja de texto de busqueda > Events > focusGained



Codifique dentro del método recién creado el código que aparece (ojo no repita lo que está sobredeado en gris)

```
private void TXTBusquedaFocusGained(java.awt.event.FocusEvent evt) {
    if (TXTBusqueda.getText().equals("Escriba texto a buscar")){
        TXTBusqueda.setText("");
    }
}
```

```
private void TXTBusquedaFocusLost(java.awt.event.FocusEvent evt) {
    if (TXTBusqueda.getText().isEmpty()){
        TXTBusqueda.setText("Escriba texto a buscar");
    }
}
```

De igual manera proceda pero con focusLost, Clic derecho sobre la caja de texto > Events > focusLost

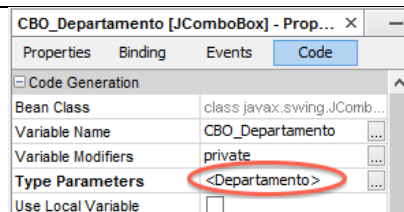
Puede aplicar lo mismo pero solo a los JTextField

El siguiente código es de ejemplo si necesita usar un combo box en algún momento con datos por ejemplo cuando use llaves foráneas:

Agregue el comboBox de la paleta y ponga el "variable name" que inicie el nombre con CBO ej CBODepartamento

Luego en Propiedades de clic en la pestaña Code (sombreada en Azul) y donde dice "Type Parameters" ponga "<Departamento>".

Esto es para que el combo solo sea de Departamentos.





<pre>private javax.swing.JComboBox<Departamento> CBO_Departamento; // End of variables declaration private void llenarComboDepartamento(){ Departamento unDepartamento = new Departamento(); Iterator<Departamento> itDepartamento = unDepartamento.listar(); CBO_Departamento.removeAllItems(); Departamento seleccion = new Departamento(); seleccion.setNombreDepartamento("Seleccione Departamento"); CBO_Departamento.addItem(seleccion); while (itDepartamento.hasNext()) { CBO_Departamento.addItem(itDepartamento.next()); } }</pre>	Ahora cree un método que se llame <code>llenarComboDepartamento()</code> con el código que aparece en el pantallazo. (ojo no escriba lo que esta sombreado con gris)
---	--

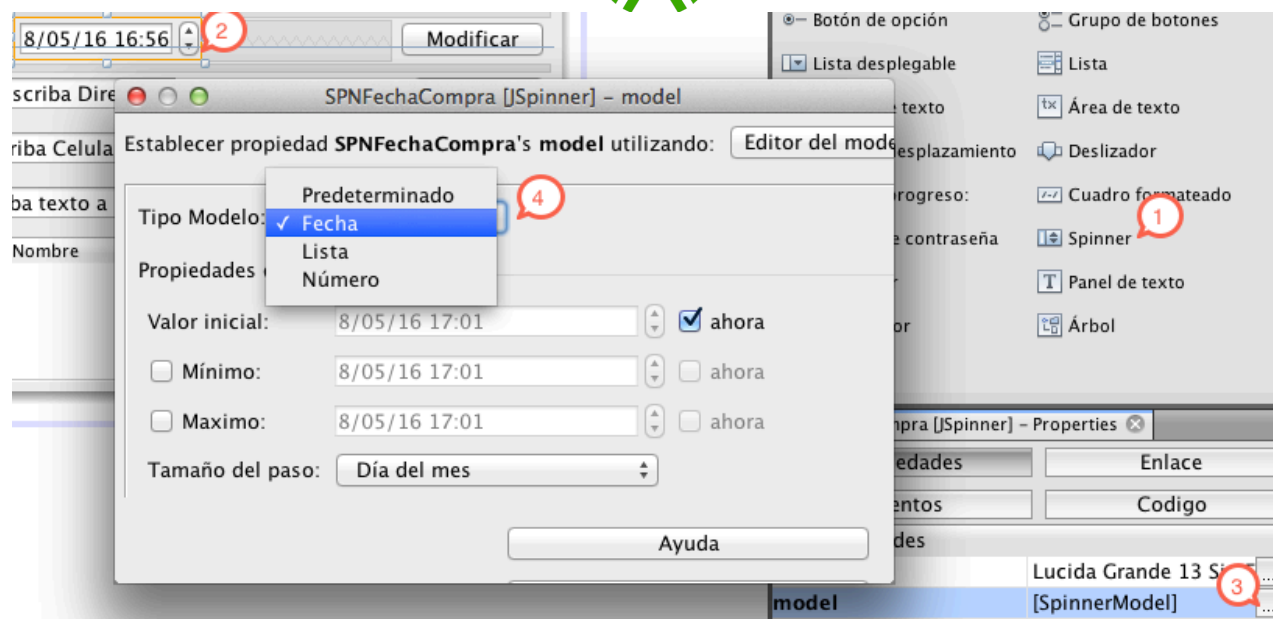
y el metodo de obtener que incluye la forma de obtener datos del JComboBox y ademas la forma de obtener la fecha de un JSpinner configurado para mostrar fechas.

```
private Compra obtenerCompra(){  
    Compra laCompra = new Compra();  
    laCompra.setIdCompra((Integer) TXTIdCompra.getValue());  
    laCompra.setIdClienteCompra(((Cliente) CBO_Departamento.getSelectedItem()).getIdCliente());  
    laCompra.setFechaCompra(new java.sql.Date(((java.util.Date) SPNFechaCompra.getValue()).getTime()));  
    return laCompra;  
}
```

Deberia también modificar el metodo <code>llenarTabla</code> y <code>llenarTablaConBusqueda</code> fijese en donde dice <code>filaCompra[1]</code> es para cuando use JComboBox	<pre>unaCompra = itCompra.next(); filaCompra[0] = unaCompra.getIdCompra(); filaCompra[1] = new Cliente().buscarPorId(unaCompra.getIdClienteCompra()); filaCompra[2] = unaCompra.getFechaCompra();</pre>
---	---

<pre>if (fila>-1){ TXTIdCompra.setValue((Integer) TBLCliente.getValueAt(fila, 0)); CBOCliente.setSelectedItem((Cliente) TBLCliente.getValueAt(fila, 1)); SPNFechaCompra.setValue((java.util.Date) TBLCliente.getValueAt(fila, 2)); BTNModificar.setEnabled(true); }</pre>	En TBL MouseClicked modificar la fila que contiene el comboBox (si necesita también el manejo de fechas en el pantallazo aparece como cargar fechas)
--	--

Tip: Para el manejo de fechas puede usar un Jspinner y en el model lo configura a Date.



En el método limpiarFormulario() puede agregar el código para reinicializar la fecha:

```
SPNFechaCompra.setModel(new javax.swing.SpinnerDateModel());
```

4. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN EN EL PROCESO FORMATIVO.

Realice el MVC para su proyecto formativo, para ello recuerde los pasos:

1. Crear un proyecto sin clase principal.
2. Crear los tres paquetes (modelo – vista – controlador)
3. Crear la ConexionBD en el paquete “modelo”
4. Crear el MDI en el paquete “vista” y renombrar el fondo negro con “escritorio”.
5. Crear los modelos, controlares y vistas (JInternalFrame) necesarios.

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Ejecución	AP02 CODIFICAR LOS MÓDULOS DEL SISTEMA DE INFORMACIÓN.	AA08 Codificar los módulos del software	Evidencias de Producto: Algoritmos implementados en java	Crea la base de datos en el motor de base de datos seleccionado, siguiendo	Valoración de Producto Lista de verificación



		stand-alone, web y móvil		especificaciones técnicas del informe, según normas y protocolos de la empresa.	
--	--	--------------------------	--	---	--

5. GLOSARIO DE TÉRMINOS

Array: También llamado vector, es un conjunto de datos a los cuales se puede acceder por su índice el cual comienza en uno ej. String[] nombres = {"Indigo", "Terracota", "Cyan", "Magenta", "Verde"}; el nombre[2] será "Cyan".

Atributo: Es una variable que se declara al nivel de la clase no por dentro de los métodos y puede ser usada en cualquier método de la clase.

Método: Es un conjunto de declaraciones de variables y operaciones envueltas dentro de una función que puede ser llamada dentro de la misma clase o dependiendo del nivel de acceso desde otra clase.

6. REFERENTES BIBLIOGRÁFICOS

Kenneth E. Kendall, Julie E. Kendall, Análisis y Diseño de Sistemas 6ª edición, 2005, Pearson Education.

Ian Sommerville, Ingeniería del Software 7ª edición, 2005, Pearson Education.

Kai Qian, Xiang Fu, Lixin Tao, Chong Wei, Jorge Díaz, Software Architecture and Design Illuminated, 2010, Jones and Barlett publishers.

7. CONTROL DEL DOCUMENTO

	Nombre			Cargo	Dependencia	Fecha
Autor (es)	Germán Vargas	Alberto	Cruz	Instructor	Coordinación Académica	15 de febrero 2017

8. CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre			Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)	Miguel Ángel Mendoza	Ángel	Tejedor	Instructor	Coordinación Académica	25 de noviembre 2024	Se actualizó al formato actual.